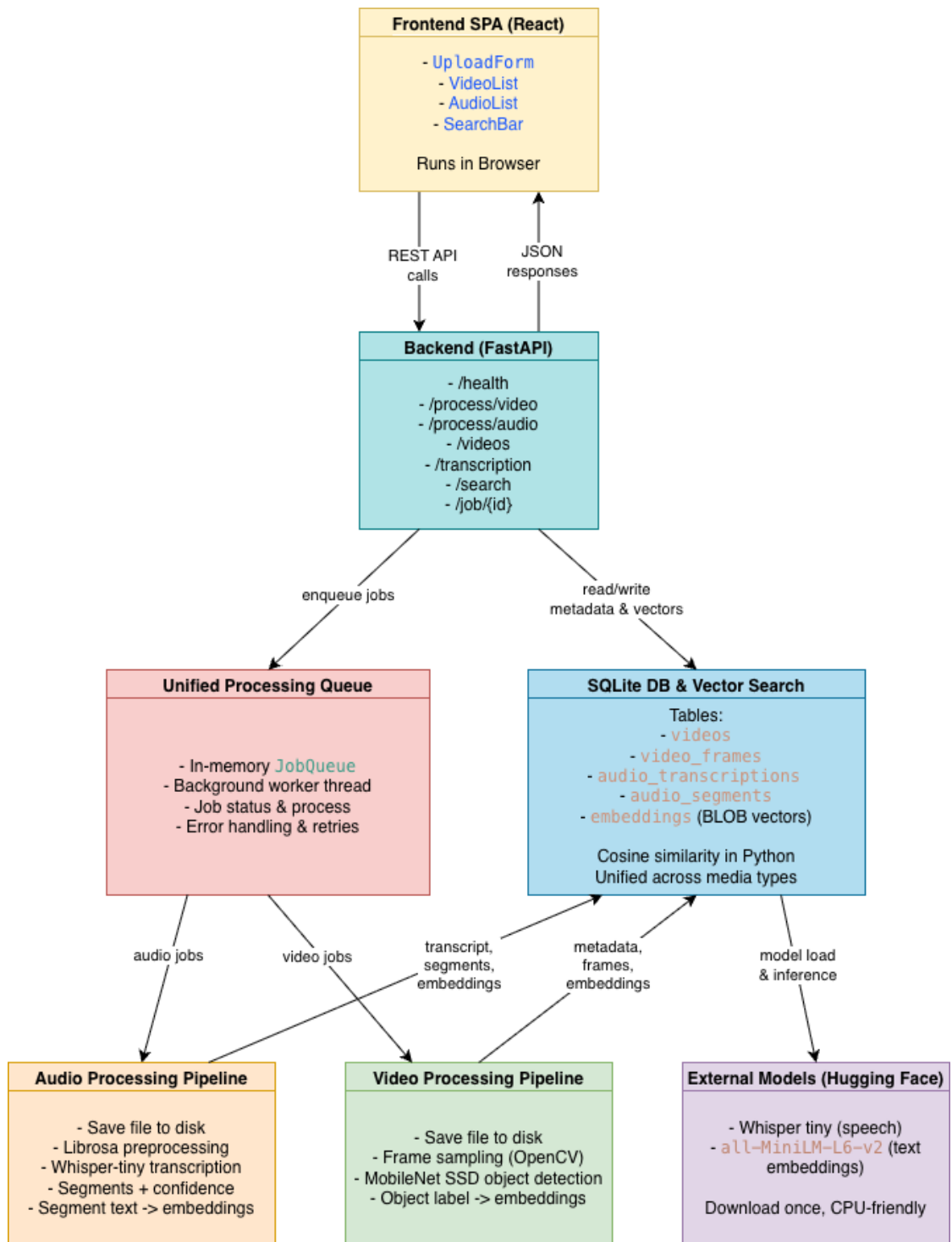


Diagram



- **Frontend (React SPA)**
 - Modules: `UploadForm`, `VideoList`, `AudioList`, `SearchBar`
 - Talks to backend via REST (`/process/*`, `/videos`, `/transcriptions`, `/search`, `/jobs/{id}`)
- **Backend (FastAPI)**
 - API Layer:
 - `/health`
 - `/process/video`, `/process/audio`
 - `/jobs/{id}`
 - `/videos`

- /transcriptions
 - /search
- **Processing Queue**
 - JobQueue + background worker thread
 - Jobs: video or audio processing
- **Video Processing Pipeline**
 - Save upload to disk
 - OpenCV frame sampling
 - MobileNet SSD object detection
 - Key frame saving
 - Object labels → embeddings (SentenceTransformers)
 - Save Video, VideoFrames, Embeddings to SQLite
- **Audio Processing Pipeline**
 - Save upload to disk
 - Librosa load + chunk
 - Whisper tiny transcription
 - Segments + confidence
 - Segment text → embeddings
 - Save AudioTranscription, AudioSegments, Embeddings to SQLite
- **Vector Search**
 - Embeddings table (BLOB)
 - Cosine similarity in Python (search_by_vector)
 - Unified across video_object and audio_segment
- **Database**
 - SQLite
 - Tables: Video, VideoFrame, AudioTranscription, AudioSegment, Embedding
- **External Services**
 - Hugging Face models pulled once (Whisper + sentence transformers)
 - No external online services at runtime (offline friendly)

Explanation

- **Vector similarity approach**
 - Chosen **Option 3** (embeddings as BLOB in SQLite + cosine similarity in Python).
 - Pros: simple, no extra infra, works on CPU, same representation across video & audio.
 - Trade-offs: slower for very large datasets; could move to FAISS later.
- **Handling larger files**
 - Video:
 - Down-sampling via frame interval and MAX_FRAMES_PER_VIDEO.
 - CPU-friendly MobileNet SSD.
 - Audio:
 - Chunked processing (AUDIO_CHUNK_SECONDS) to avoid OOM.
 - Streaming-like loop updates job progress.
 - Both:
 - Background queue so frontend doesn't block; progress polled via /jobs/{id}.
- **Scalability & improvements with more compute**
 - Move from in-process queue to Celery/RQ + Redis.
 - Use GPU-enabled Docker base image for Whisper & detection.
 - Replace cosine-in-Python with FAISS index or vector DB.
 - Shard media and embedding tables, add proper indices.
- **Processing speed vs accuracy**
 - Whisper tiny + MobileNet SSD = good speed, acceptable accuracy for demo.
 - With more resources:
 - Upgrade to Whisper base/small.
 - Use stronger detectors (YOLOv8n/s) and better sampling strategies.
 - Current design makes models pluggable (just swap processing functions).